

Lab 5 — Maleabilidad de CBC (bit-flipping)

Vulnerabilidad: el cifrado sin autenticar es *maleable*: un atacante puede modificar el texto cifrado para provocar cambios **predecibles** en el texto claro, sin conocer la clave. En CBC el descifrado del primer bloque es $P_0 = D(K, C_0) \oplus IV$. Como el IV se combina con XOR, voltear un bit del IV voltea el mismo bit del texto claro. Esta maleabilidad es la base de ataques mayores, como el **padding oracle** (POODLE, Lucky13, el fallo de ASP.NET de 2010).

El reto

El token cifrado descifra a `admin=0;alumno=Juan;grupo=fresas`. Consigue que descifre a `admin=1;...` **sin tocar la clave**.

LABORATORIO · AVANZADO

5 · Maleabilidad de CBC (bit-flipping)

CBC NO AUTENTICA: SE PUEDE ALTERAR EL TEXTO CLARO SIN LA CLAVE

OBJETIVO

Cambiar el mensaje descifrado sin conocer la clave, manipulando solo el IV.

EL RETO

El token cifrado descifra a «admin=0;...». Tu meta: que descifre a «admin=1;...» SIN tocar la clave. En CBC, $P_0 = D(C_0) \oplus IV$, así que al voltear un byte del IV volteas el mismo byte del texto claro. La rama de arriba calcula $IV \oplus \text{máscara}$ (la máscara ya tiene 0x01 en la posición 6). Copia ese «IV modificado» al parámetro IV del bloque «AES Decrypt» de abajo y observa el resultado.

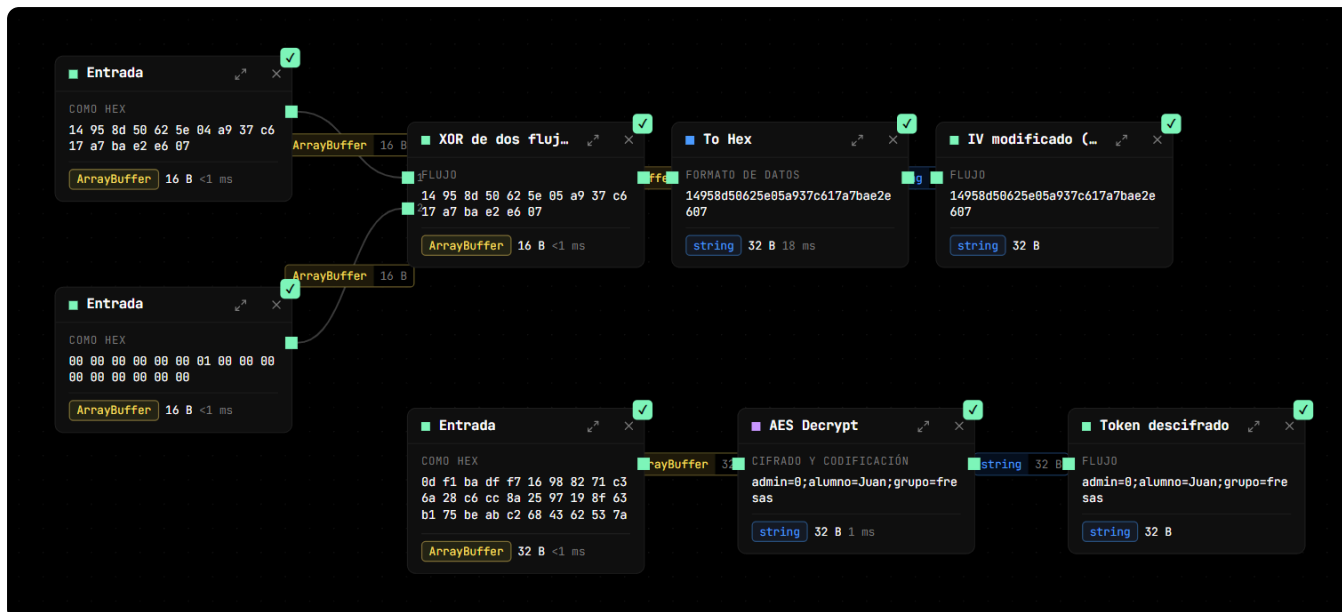
PISTAS (4)

LO LOGRASTE CUANDO...

La salida descifrada empieza por «admin=1».

VER SOLUCIÓN (WRITEUP)

AL LIENZO →

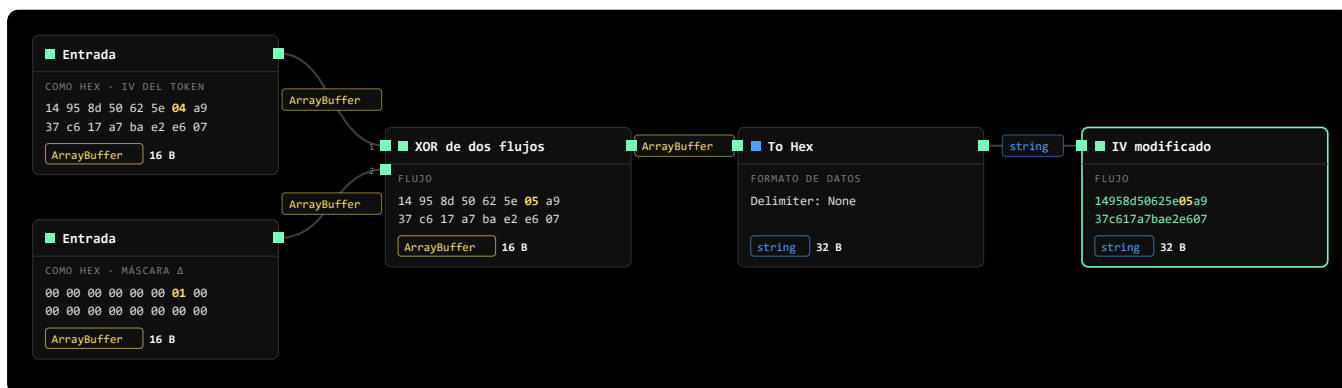


Solución

El objetivo es el byte en la posición 6 del texto claro: '0' (0x30) debe volverse '1' (0x31).

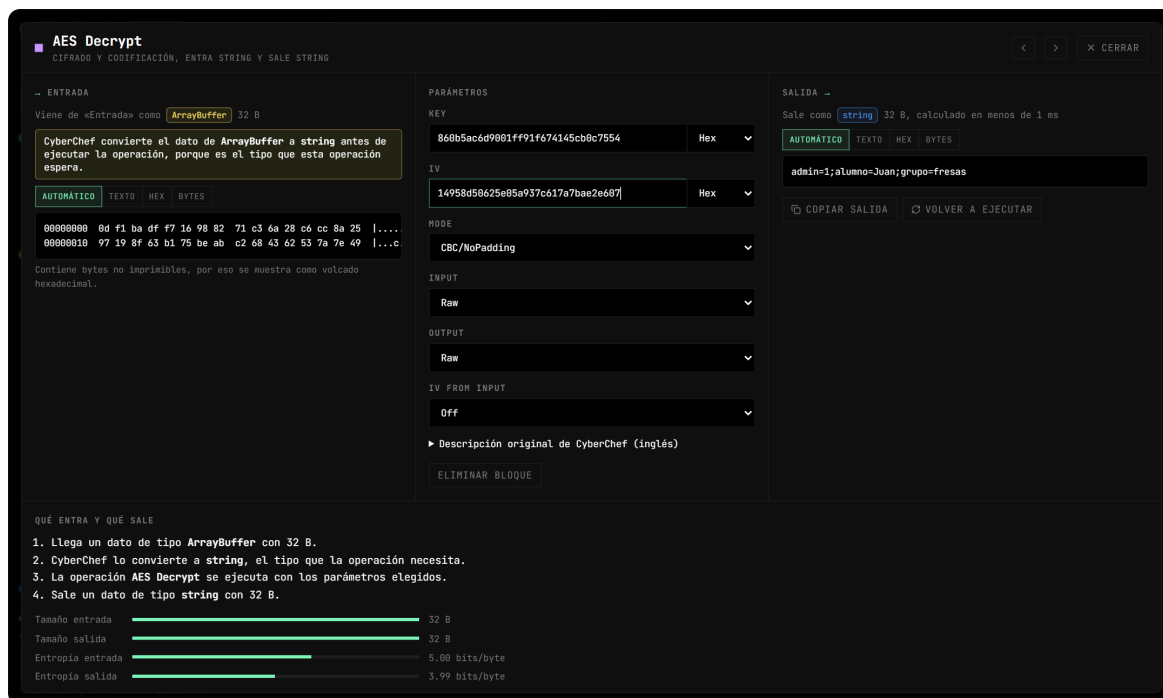
$$\Delta = '0' \oplus '1' = 0x30 \oplus 0x31 = 0x01$$

Como ese byte está en el primer bloque, se manipula el **IV** en esa misma posición. La rama superior del flujo ya lo calcula:



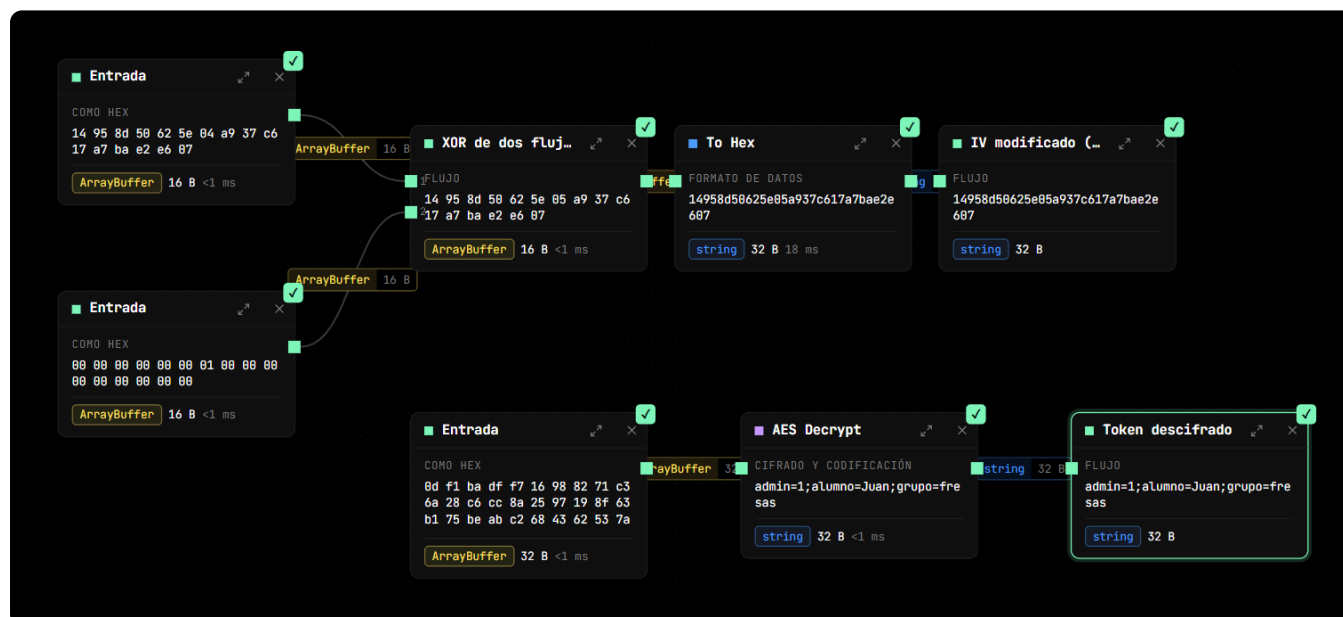
$IV' = IV \oplus \text{máscara}$ cambia solo el byte 6 del IV (resaltado): 04 pasa a 05. El resto del IV queda igual.

El paso del reto: copia el valor de *IV modificado* (botón **Copiar salida** en su detalle) y pégalo en el parámetro **IV** (formato Hex) del bloque *AES Decrypt* de abajo:



El resultado pasa a:

admin=1;alumno=Juan;grupo=fresas



Solo cambió `admin=0` → `admin=1`. El resto del mensaje queda intacto, porque el IV únicamente afecta al primer bloque.

El bloque de descifrado tiene la clave solo para que **veas** el efecto. El atacante real no la conoce ni la necesita: manipula el mensaje a ciegas, confiando en que sabe dónde está el byte objetivo.

Por qué

En CBC:

$$P_0 = D(K, C_0) \oplus IV$$
$$P'_0 = D(K, C_0) \oplus IV' = D(K, C_0) \oplus (IV \oplus \Delta) = P_0 \oplus \Delta$$

$D(K, C_0)$ no cambia porque ni la clave ni C_0 se tocan. El XOR con Δ se traslada directamente al texto claro:

CBC, primer bloque: $P_0 = D(K, C_0) \oplus IV$
Voltear un bit del IV voltea el mismo bit de P_0 . $D(K, C_0)$ no cambia: ni la clave ni C_0 se tocan.

POSICIÓN	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$D(K, C_0)$ lo fija el cifrado	75	f1	e0	39	0c	63	34	92	56	aa	62	ca	d4	8d	db	4d
IV original del token	14	95	8d	50	62	5e	04	a9	37	c6	17	a7	ba	e2	e6	07
P_0 texto claro	a	d	m	i	n	=	0	;	a	l	u	m	n	o	=	j

El atacante XORea el IV con la máscara $\Delta = '0' \oplus '1' = 0x01$ en la posición 6:

$D(K, C_0)$ igual que antes	75	f1	e0	39	0c	63	34	92	56	aa	62	ca	d4	8d	db	4d
$IV' = IV \oplus \Delta$ $04 \rightarrow 05$	14	95	8d	50	62	5e	05	a9	37	c6	17	a7	ba	e2	e6	07
P'_0 sin conocer K	a	d	m	i	n	=	1	;	a	l	u	m	n	o	=	j

$P'_0 = P_0 \oplus \Delta \rightarrow$ «admin=0» pasa a «admin=1»; los demás bytes quedan intactos.

Los valores son los reales del lab. En la posición 6, $D(K, C_0) = 0x34$; con el IV original $0x34 \oplus 0x04 = 0x30$ ('0'); con el IV modificado $0x34 \oplus 0x05 = 0x31$ ('1').

Si el byte objetivo estuviera en el bloque n , se voltearía el mismo byte de C_{n-1} en su lugar (a costa de convertir el bloque $n-1$ en basura, un compromiso habitual del ataque).

Verifícalo con otras herramientas

Con `openssl`, `xxd` y `bash` (ver [requisitos](#)). Descifra el token original:

```
K=860b5ac6d9001ff91f674145cb0c7554
IV=14958d50625e04a937c617a7bae2e607
echo 0df1badff716988271c36a28c6cc8a2597198f63b175beabc2684362537a7e49 | xxd -r -p > token.bin
openssl enc -d -aes-128-cbc -K $K -iv $IV -nopad -in token.bin
# admin=0;alumno=Juan;grupo=fresas
```

Calcula $IV' = IV \oplus \Delta$ en el byte 6 (caracteres 12–13 de la cadena hex) y descifra con él:

```

IV2=$(printf '%s%02x%s' ${IV:0:12} $((0x${IV:12:2} ^ 0x01)) ${IV:14})
echo $IV2
# 14958d50625e05a937c617a7bae2e607
openssl enc -d -aes-128-cbc -K $K -iv $IV2 -nopad -in token.bin
# admin=1;alumno=Juan;grupo=fresas

```

¿Por qué `-nopad`? El token son exactamente 32 bytes sin relleno PKCS#7 (en CipherFlow, modo CBC/NoPadding). Si lo quitas, `openssl` busca un relleno válido al final, no lo encuentra y responde `bad decrypt`. Justo esa diferencia entre «relleno válido» y «relleno inválido», si un servidor la deja ver, es el oráculo del ataque de la sección siguiente.

Reto extra: cambiar el segundo bloque. Para tocar bytes del bloque 1 hay que voltear el mismo byte de `C0`. Así se cambia `grupo=fresas` por `grupo=mangos` (posiciones 26–31, es decir, 10–15 dentro del bloque 1):

```

python3 -c "c=bytearray(open('token.bin','rb').read());d=bytes(a^b for a,b in zip(b'fresas',b'mangos'));c[10:16]=bytes(x^y for x,y in zip(c[10:16],d));open('token2.bin','wb').write(c)"
openssl enc -d -aes-128-cbc -K $K -iv $IV -nopad -in token2.bin | xxd
# 00000000: 0d2e 5898 d49d c2ae 7f47 ddfb 4a1c 232f  ..X.....G..J.#/    ← bloque 0 destruido
# 00000010: 7561 6e3b 6772 7570 6f3d 6d61 6e67 6f73  uan;grupo=mangos    ← bloque 1 a la carta

```

El bloque 1 queda exactamente como quería el atacante, pero el bloque 0 se convierte en basura: ese es el compromiso del que habla la sección anterior.

La mitigación, comprobada. `openssl enc` no admite modos AEAD (con `-aes-128-gcm` responde `AEAD ciphers not supported`), así que usa Node, que ya necesitas para CipherFlow. Guarda esto como `gcm.js` y ejecútalo con `node gcm.js`:

```

const crypto = require('node:crypto')
const key = Buffer.from('860b5ac6d9001ff91f674145cb0c7554', 'hex'), iv = crypto.randomBytes(12)
const enc = crypto.createCipheriv('aes-128-gcm', key, iv)
const ct = Buffer.concat([enc.update('admin=0;alumno=Juan;grupo=fresas'), enc.final()] ), tag = enc.getAuthTag()
const abrir = c => { const d = crypto.createDecipheriv('aes-128-gcm', key, iv); d.setAuthTag(tag); return Buffer.concat([d.update(c), d.final()]).toString() }
console.log('original :', abrir(ct))
const mod = Buffer.from(ct); mod[6] ^= 0x01 // el mismo volteo de bit que en CBC
try { console.log('alterado :', abrir(mod)) } catch (e) { console.log('alterado :', e.message) }

```

```

original : admin=0;alumno=Juan;grupo=fresas
alterado : Unsupported state or unable to authenticate data

```

GCM cifra en modo contador, así que el volteo sí cambiaría `'0'` por `'1'` en el texto claro, pero la etiqueta ya no coincide y el mensaje se rechaza antes de entregarlo.

Del bit-flipping al padding oracle

Este lab muestra el *primitivo*. El **padding oracle** lo convierte en un descifrado completo: si el servidor revela (por un mensaje de error o por el tiempo de respuesta) si el relleno PKCS#7 de un mensaje es válido, el atacante ajusta byte a byte el bloque anterior hasta forzar un relleno válido y, de ahí, deduce $D(K, C_i)$ y por tanto el texto claro —sin la clave—. Es adaptativo (cientos de consultas por bloque), por eso no se resuelve en un grafo estático como este; pero descansa exactamente en la maleabilidad que acabas de explotar.

Impacto real

- **POODLE (2014)**: padding oracle sobre SSL 3.0 con CBC.
- **Lucky Thirteen (2013)**: oracle por tiempo en TLS.
- **ASP.NET (2010)**: padding oracle que permitía leer archivos y forjar cookies.
- **Cookies de sesión** cifradas con CBC sin MAC: bit-flipping para escalar privilegios, justo como este

`admin=0` → `admin=1`.

Mitigación

Usa **cifrado autenticado (AEAD)**: AES-GCM o ChaCha20-Poly1305. Añaden una etiqueta que se verifica antes de descifrar, así que cualquier bit alterado se detecta y el mensaje se rechaza. Pruébalo en CipherFlow: cifra un texto con *AES Encrypt* en modo **GCM** (Output `Hex`); la salida trae el cifrado y, debajo, una línea `Tag: ...`. Pasa ese cifrado a *AES Decrypt* en modo GCM (Input `Hex`) con el tag en el parámetro **GCM Tag** y descifra bien. Ahora cambia un solo dígito del cifrado o del IV: en vez de entregar un texto claro manipulado, el bloque falla con *Unable to decrypt input with these parameters*. Si no puedes usar AEAD, aplica **encrypt-then-MAC** con un MAC de tiempo constante.